WEEKEND ENGINEERING ASSIGNMENT

15-Day Intern Performance Evaluation | Weekend 2 | HR distribution copy

Purpose. This second-weekend assignment assesses independent project understanding, architecture, database and API design, technical judgment, validation, commercial awareness and ownership. **All five interns must work independently on BOTH projects.** These are separate practice projects, not their weekday client assignments.

1. Projects for this weekend

Project	Minimum functional scope
AI SaaS Workspace	Authentication and workspace access; document upload; AI-assisted questions/answers grounded in uploaded content; request history; per-user usage limits; clear handling of unsupported answers and service failures.
Appointment & Scheduling SaaS	Provider/service profiles; working hours and available slots; customer bookings; rescheduling and cancellation; booking status; reminders (mock provider permitted); prevention of overlapping bookings.

Scope: For each project, deliver a coherent runnable MVP with at least one complete user journey. Simulated billing, notifications, or AI provider calls are acceptable only when clearly labelled and their proposed production integration is explained. Do not claim planned features are implemented.

2. Mandatory work for EVERY project

• **Requirements and user flows:** Describe users, roles, goals, scope, assumptions, acceptance criteria, exceptions and at least one complete end-to-end journey. Explain why the product exists and what is out of scope.

• **Architecture:** Create a Lucidchart system architecture showing frontend, API, authentication, database, object storage where relevant, background jobs, external services, security boundaries and deployment location. Explain component responsibilities and design trade-offs.

• **Database design:** Provide a Lucidchart ERD and executable schema.sql or migrations, including PK/FK, field types, constraints, indexes, relationships and data-integrity rules. For scheduling, address double bookings and concurrent requests; for AI SaaS, address file ownership and request usage records.

• **API workflows:** Create Lucidchart sequence/API flows for a successful journey and at least one failure journey. Document routes, methods, auth, request/response examples, validation, errors and idempotency/retry behavior where relevant.

• **Implementation:** Provide a runnable repository with documented setup/run/test instructions, environment variable names (no credentials), and a 3–5 minute product walkthrough. Explain implemented versus mocked services.

• **Developer-side QA:** Write and execute own positive, negative, boundary, authorization, duplicate/retry and regression test cases. Capture expected versus actual results, defect references and evidence of fixes.

• **Security and reliability:** Cover role-based access, tenant/user data isolation as appropriate, file upload controls, secret management, failure recovery, logging, monitoring, backups and background-job retry safety.

• **Deployment and costs:** Compare at least two realistic hosting choices such as VPS/Hostinger and AWS, addressing compute, database, storage, backups, bandwidth, monitoring, queues, domain and paid APIs. Include region, currency, estimated monthly spend, assumption, vendor pricing URL and date accessed. No invented prices.

• **Scaling:** Present capacity and architecture adjustments for 100, 1,000, 10,000 and 1,000,000 users. Define active usage/request assumptions; identify bottlenecks, caching/queues, database capacity and cost implications.

• **Project-specific depth:** AI SaaS: model/API versus open-source choice, RAG/retrieval design, chunking, citations, token limits, prompt injection, inference/GPU cost and failure fallbacks. Scheduling SaaS: slot generation, time zones/DST, concurrent reservations, cancellation policy, reminder scheduling and retry behavior.

• **Source and AI disclosure:** Original code or open-source adaptation is allowed. List sources, repositories, licenses, reused code and modifications. AI assistance is discouraged; if used, disclose its purpose and explain independent verification. Every submission must be defensible in the Monday interview.

• **Submission and interview:** Submit the Weekend2_YourName Drive folder containing 01_AI_SaaS and 02_Scheduling_SaaS, with working reviewer access. Share links with HR and your assigned Junior 1/2 reviewer before Monday's interview. The same reviewer examines architecture, code, testing, trade-offs and costs.

3. Exact deliverable formats — for EACH project

File / output	Required format and contents
01 - README.md	Project purpose, stack, scope, assumptions, user flows, setup/run/test commands and known gaps.
02 - architecture.pdf	Lucidchart export with components, data flows, trust/deployment boundaries, plus editable Lucidchart URL.
03 - database-design.pdf	Lucidchart ERD with PK/FK, data types, cardinality, constraints and indexes; include schema.sql or migrations and editable URL.
04 - api-workflows.pdf	Lucidchart sequence/API flows covering success and failure; include api-spec.md or openapi.yaml.
05 - test-cases.xlsx	Test ID, Feature, Preconditions, Steps, Input, Expected, Actual, Status, Defect ID, Evidence Link; execution proof.
06 - engineering-decisions.md	Alternatives, chosen approach, reasoning, technical risks, security, reliability, constraints, implemented vs proposed.
07 - source + demo	GitHub repository, instructions, relevant commits, runnable MVP and accessible 3–5-minute demo.
08 - cost-and-scaling.xlsx	Side-by-side vendor cost breakdown with quantities, unit pricing, monthly totals, source URLs/dates; user-scale scenarios. Include model/token costs for AI SaaS.
09 - sources-and-disclosure.md	Third-party technical/pricing sources, open-source URLs/licenses/reuse details and AI-use disclosure or "None."
10 - submission-index.md	All deliverable links for both projects, editable diagrams, exports, test evidence, repo/demo and missing items.

Suggested folder layout: Weekend2_YourName / 01_AI_SaaS / [01–09 files] and 02_Scheduling_SaaS / [01–09 files], with 10_submission-index.md at the parent-folder level. Keep diagrams editable; submit PDF exports as well.

4. Monday interview and performance evaluation

Evaluation dimension	Weight
Problem understanding	20%
Architecture and design	25%
Implementation quality	25%
Testing and edge cases	20%
Explanation and independent ownership	10%

Each project is scored separately from 1–5 for each dimension. Weighted percentage = sum of (rating / 5 x weight). Reviewers may request a live walkthrough, query about design alternatives and costs, ask for a bug to be diagnosed, or request a small change. Scores contribute to the broader **15-day performance evaluation**; copied diagrams or working code without understanding are not sufficient.